Sécurité Web

Failles principales et bonnes pratiques

Ressources de veille :

→ **https://owasp.org/** — référence mondiale sur les failles web
→ **https://hub.docker.com/_/postgres/tags** — suivre les versions des images Docker

Vue d'ensemble des priorités

Les failles web sont classées selon leur criticité. Voici les trois niveaux à connaître :

⚠️ PRIORITÉS

- Broken Access Control
- Injection SQL
- XSS — Cross-Site Scripting
- Chiffrement et hachage
- CSRF
- Contraintes de mot de passe

● IMPORTANTS

- HTTP / HTTPS — MitM
- Variables d'environnement et clés API
- Validation et échappement des entrées
- CORS

○ BONUS

- Microservices
- DDoS
- Dépendances
- Utilisateurs de base de données

⚠️ Broken Access Control

Problème

Des routes sensibles sont accessibles via l'URL sans vérification des droits.

Solution

Mettre en place un middleware d'authentification sur toutes les routes protégées, et vérifier les rôles/permissions côté serveur.

⚠ Le contrôle d'accès brisé est classé **#1** dans le Top 10 OWASP. C'est la faille la plus répandue et la plus critique à corriger en priorité.

⚠️ Injection SQL

Problème

Un attaquant exécute du code SQL malveillant (ex. `DROP DATABASE`) via un formulaire ou une requête envoyée au serveur.

Solution

Utiliser exclusivement des requêtes préparées/paramétrées. Ne jamais concaténer directement les entrées utilisateur dans une requête.

⊗ La concaténation directe d'entrées utilisateur dans une requête SQL est une pratique **absolument interdite** en production.

⚠️ XSS — Cross-Site Scripting

Le Cross-Site Scripting est l'une des failles les plus courantes sur le web. Elle permet à un attaquant d'injecter du code malveillant dans les pages vues par d'autres utilisateurs.

Problème

Un script potentiellement malveillant est injecté dans une page et s'exécute dans le navigateur des autres utilisateurs.

Solution

- Échapper systématiquement les données avant affichage
- Utiliser `innerText`/`textContent` plutôt que `innerHTML`
- Configurer les headers CSP (Content Security Policy)

⚠️ Chiffrement et hachage

Des données sensibles stockées ou transmises en clair constituent une faille majeure. Il est essentiel de distinguer deux approches complémentaires :

🔐 Chiffrement

Chiffrer les données dont la valeur originale doit rester exploitable (données personnelles, etc.).

Le chiffrement est **réversible** : on peut retrouver la valeur d'origine avec la clé.

🔒 Hachage

Hacher les données qui n'ont pas besoin d'être restituées (mots de passe) : le hachage est **irréversible**.

Même en cas de fuite, la valeur originale ne peut pas être retrouvée.

⚠ **Problème :** Des données sensibles sont stockées ou transmises en clair.

⚠️ CSRF — Cross-Site Request Forgery

Problème

Un site tiers déclenche une requête (formulaire ou fetch) à l'insu de l'utilisateur en exploitant sa session ou ses cookies.

Solution

- Utiliser un token CSRF par formulaire
- Configurer correctement les cookies

⚠️ Paramètres cookie recommandés

La configuration des cookies est un élément clé de la défense contre le CSRF et d'autres attaques. Voici les paramètres recommandés :

Expiration courte

Limiter la durée de vie du cookie réduit la fenêtre d'exploitation en cas de vol de session.

`httpOnly`

Non accessible depuis le JavaScript client. Protège contre le vol de cookie via XSS.

`SameSite=Strict`

Seul le domaine émetteur peut transmettre le cookie. Protège contre le CSRF.

`Secure=true`

HTTPS uniquement. Le cookie ne sera jamais transmis sur une connexion non chiffrée.

ℹ️ Ces quatre paramètres doivent être combinés pour une protection optimale des cookies en production.

⚠️ Contraintes de mot de passe

Problème

Des mots de passe trop faibles facilitent les attaques par force brute ou par dictionnaire.

Solution

Appliquer des critères basés sur les recommandations de l'ANSSI. Recommander à l'utilisateur de ne pas baser son mot de passe sur des données personnelles et de ne pas le réutiliser sur d'autres services.

Longueur minimale

Un mot de passe long est exponentiellement plus difficile à craquer par force brute.

Complexité

Mélanger majuscules, minuscules, chiffres et caractères spéciaux selon les recommandations ANSSI.

Pas de données personnelles

Ne pas baser son mot de passe sur des données personnelles (nom, date de naissance, etc.).

Pas de réutilisation

Ne pas réutiliser le même mot de passe sur d'autres services.

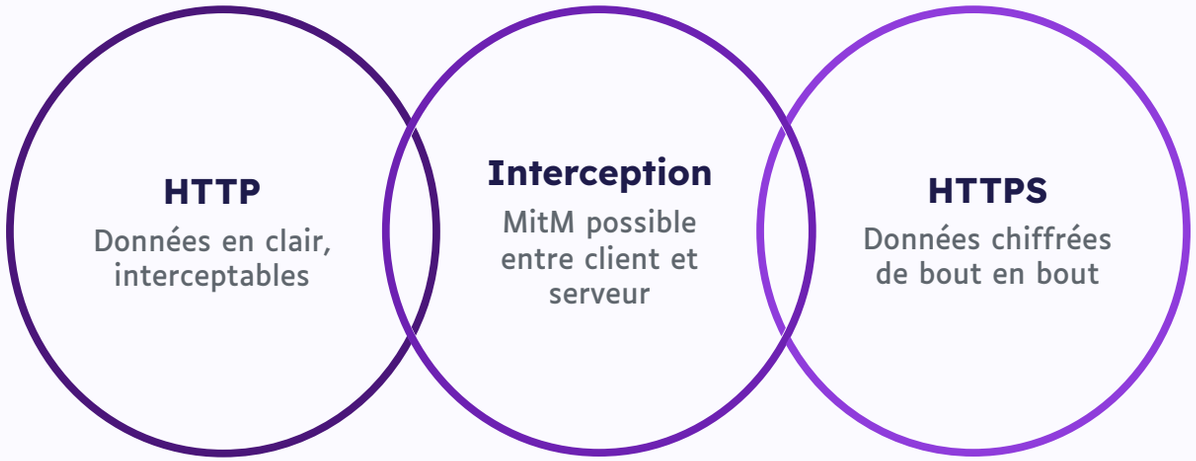
● HTTP / HTTPS — Man in the Middle (MitM)

Problème

Les informations échangées en HTTP circulent en clair et peuvent être interceptées entre l'utilisateur et le serveur.

Solution

Utiliser HTTPS pour que toutes les communications soient chiffrées de bout en bout (client → serveur).



Le passage à HTTPS est aujourd'hui un prérequis absolu pour toute application web exposée au public.

● Variables d'environnement et clés API

Problème

Des secrets sont exposés publiquement : poussés sur GitHub ou accessibles côté front-end.

Solution

- Ajouter les fichiers sensibles au `.gitignore` dès l'initialisation du projet
- Ne jamais écrire les clés API en dur dans le code
- Les utiliser exclusivement côté serveur

⊗ Une clé API exposée sur un dépôt public peut être exploitée en quelques minutes par des bots qui scannent GitHub en permanence. **Ne jamais commettre de secrets dans le code source.**

Validation et échappement des entrées

Problème

Les utilisateurs peuvent soumettre des données non conformes ou dangereuses.

Solution

- Valider toutes les entrées via un schéma côté serveur (ex. `Zod`), idéalement doublé d'une validation côté client
- Échapper les données : supprimer les balises HTML, encoder les caractères spéciaux

1

Entrée utilisateur

Données brutes soumises via formulaire ou requête

2

Validation (Zod)

Vérification du schéma côté serveur et côté client

3

Échappement

Suppression des balises HTML, encodage des caractères spéciaux

4

Données sûres

Prêtes à être traitées ou stockées

● CORS — Cross-Origin Resource Sharing

Problème

N'importe quel domaine peut interroger l'API et exploiter les données retournées.

Solution

Définir une liste blanche (whitelist) des origines autorisées côté serveur. Ne jamais utiliser `*` en production.

⊗ **Ne jamais utiliser le symbole "*" pour les origines en production**, sauf si API ouverte et partagée au public.

◯ Microservices

L'architecture microservices est une approche de conception qui décompose une application en services indépendants. Elle présente des avantages et des inconvénients à bien peser :

→ Avantages

- Scalabilité
- Testabilité
- Maintenabilité (séparation des responsabilités)
- Haute disponibilité

→ Inconvénients

- Mise en place complexe
- Multiplication des canaux de communication
- Centralisation des logs obligatoire

◯ DDoS — Distributed Denial of Service

Problème

Une quantité massive de requêtes sature le serveur et rend le service indisponible.

Solution

- Mettre en place un CDN/WAF (ex. Cloudflare)
- Du rate limiting
- Limiter la taille des payloads acceptés

CDN / WAF

Un réseau de distribution de contenu comme Cloudflare absorbe et filtre le trafic malveillant avant qu'il n'atteigne le serveur.

Rate Limiting

Limiter le nombre de requêtes par IP et par unité de temps pour empêcher la saturation du serveur.

Taille des payloads

Limiter la taille des payloads acceptés pour éviter les attaques par surcharge de données.

Dépendances

Problème

Des versions obsolètes peuvent contenir des vulnérabilités connues, ou introduire des breaking changes inattendus.

Solution

- Mettre à jour régulièrement les dépendances
- Faire de la veille sur les CVE
- Figer les versions majeures utilisées en production

La veille sur les **CVE (Common Vulnerabilities and Exposures)** permet d'être alerté rapidement lorsqu'une vulnérabilité est découverte dans une dépendance utilisée par votre projet.

◯ Utilisateurs de base de données

Problème

Des accès trop permissifs exposent l'ensemble de la base en cas de compromission.

Solution

Créer un rôle dédié par base de données, avec uniquement les privilèges nécessaires pour l'application.

⊗ Le `ALL PRIVILEGES` ne doit jamais être utilisé dans une application en production.

Appliquer le principe du moindre privilège : chaque composant de l'application ne doit avoir accès qu'aux ressources strictement nécessaires à son fonctionnement.

Tableau récapitulatif des failles

Faille	Problème	Solution clé
Broken Access Control	Routes sensibles accessibles sans vérification	Middleware d'authentification + vérification des rôles
Injection SQL	Exécution de code SQL malveillant	Requêtes préparées/paramétrées uniquement
XSS	Script malveillant injecté dans une page	Échappement + `textContent` + CSP
Chiffrement/Hachage	Données sensibles en clair	Chiffrer ou hacher selon le besoin
CSRF	Requête déclenchée à l'insu de l'utilisateur	Token CSRF + configuration des cookies
Mots de passe	Mots de passe trop faibles	Critères ANSSI + sensibilisation utilisateur
MitM / HTTP	Communications interceptées en clair	HTTPS obligatoire
Clés API	Secrets exposés publiquement	`.gitignore` + côté serveur uniquement
Validation entrées	Données non conformes ou dangereuses	Schéma Zod + échappement
CORS	N'importe quel domaine peut interroger l'API	Whitelist des origines, jamais `*`
DDoS	Saturation du serveur	CDN/WAF + rate limiting + taille payloads
Dépendances	Vulnérabilités dans les versions obsolètes	Mise à jour + veille CVE + versions figées
Utilisateurs BDD	Accès trop permissifs	Rôle dédié + privilèges minimaux

Ressources de veille

Pour rester à jour sur les failles de sécurité web et les bonnes pratiques, voici les ressources incontournables :

🌐 OWASP

https://owasp.org, référence mondiale sur les failles web. Publie le célèbre **Top 10** des vulnérabilités les plus critiques, mis à jour régulièrement.

🐳 Docker Hub

https://hub.docker.comsuivre les versions des images Docker pour s'assurer d'utiliser des versions à jour et non vulnérables.

🛡️ ANSSI

L'Agence Nationale de la Sécurité des Systèmes d'Information publie des recommandations officielles sur les mots de passe, la cryptographie et la sécurité des systèmes d'information.

📋 CVE Database

La base de données CVE (Common Vulnerabilities and Exposures) recense toutes les vulnérabilités connues. Indispensable pour la veille sur les dépendances.

Récapitulatif et bonnes pratiques

01

Sécuriser les accès

Middleware d'authentification, vérification des rôles, HTTPS, cookies bien configurés.

02

Protéger les données

Requêtes préparées, chiffrement, hachage des mots de passe, variables d'environnement.

03

Valider et échapper

Schéma Zod côté serveur, échappement des entrées, headers CSP, CORS avec whitelist.

04

Maintenir et surveiller

Mise à jour des dépendances, veille CVE, rate limiting, rôles BDD avec privilèges minimaux.

La sécurité web n'est pas un état final mais un **processus continu**. Consultez régulièrement **owasp.org** et restez en veille sur les nouvelles vulnérabilités.